

Verbeterde Samenwerking tussen Onderzoekers en IT via Literate Programming in .NET-Applicaties.

Bachelorproef, 2025-2026

Caradoc Roetynck

E-mail: caradoc.roetynck@student.hogent.be

Co-promotor: S. Bosch (ILVO, Samuel.Bosch@ilvo.vlaanderen.be)

Samenvatting

Wetenschappelijke rekenmodellen binnen instellingen zoals ILVO worden vaak ontwikkeld in .NET-omgevingen, waarbij de onderliggende aannames en berekeningen voor onderzoekers niet altijd transparant of gemakkelijk navolgbaar zijn. Hierdoor ontstaat een kloof tussen de onderzoekers die het model gebruiken en de ontwikkelaars die het onderhouden. Dit onderzoek richt zich op de vraag hoe een literate-programming-achtige aanpak kan worden toegepast om een bestaand .NET-rekenmodel begrijpelijker en transparanter te maken voor onderzoekers, zonder het model zelf fundamenteel te herontwerpen. Om deze centrale vraag te beantwoorden wordt eerst onderzocht welke documentatie-, workflow- en transparantieproblemen onderzoekers vandaag ervaren wanneer zij met ILVO-rekenmodellen werken, zodat duidelijk wordt waar de bestaande knelpunten liggen. Vervolgens wordt nagegaan welke technieken en tools zoals Emacs Org Mode, Jupyter Notebooks, Quarto en andere systemen voor literate programming – geschikt zijn om code, uitleg en reproduceerbare berekeningen op een geïntegreerde manier te combineren in de context van .NET-gebaseerde wetenschappelijke software. Ten slotte wordt onderzocht hoe op basis van deze inzichten een proof-of-concept workflow kan worden ontworpen en geëvalueerd die deze transparantiemethoden toepast op een bestaande .NET-codebase, met als doel onderzoekers meer inzicht en autonomie te geven bij het interpreteren en verifiëren van modelgedrag. Dit werk beoogt zo een bijdrage te leveren aan de brug tussen software-engineeringpraktijken en wetenschappelijke modelontwikkeling.

Keuzerichting: Mobile & Enterprise development

Sleutelwoorden: C#, blazor, literate programming, Domain Driven Design, Domain Specific Language

Inhoudsopgave

1	Inleiding	1
2	Literatuurstudie	2
	2.1 Literate programming	2
	2.2 Org-mode Babel	2
	2.3 Jupyter notebook	3
	2.4 RMarkdown	3
	2.5 Tussenbesluit	3
3	Methodologie	3
	3.1 Probleem- en domeinanalyse	3
	3.2 Literatuurstudie & technologieverkenning	3
	3.3 Proof-of-Concept ontwikkeling	4
	3.4 Evaluatie en validatie	4
4	Verwacht resultaat, conclusie	4
	4.1 Probleem- en domeinanalyse	4
	4.2 Literatuurstudie & technologieverkenning	4
	4.3 Proof-of-Concept ontwikkeling	4
	4.4 Evaluatie en validatie	4
	Referenties	4

1. Inleiding

Wetenschappelijke instituten zoals ILVO (Instituut voor Landbouw-, Visserij- en voedingsOnderzoek) maken intensief gebruik van complexe rekenmodellen om emissies, productie-impact en andere landbouwparameters te analyseren. Deze modellen bestaan uit ingewikkelde verzamelingen van formules die door de onderzoekers worden ontwikkeld en vervolgens geïmplementeerd worden door het IT departement in softwaretoepassingen. Aangezien de onderzoekers deze toepassingen gebruiken om hun hypothesen te testen en scenario's simuleren, is de kwaliteit en duidelijkheid van deze implementatie van groot belang. In praktijk blijken deze modellen moeilijk aan te passen zodra ze in de code zijn gegoten.

Binnen ILVO wordt dit probleem duidelijk zichtbaar in hun .NET applicaties waar emissieformules en bedrijfsparameters zijn verwerkt in de C# logica. Onderzoekers beschikken over de nodige inhoudelijke expertise om hun formules en parameters te wijzigen, maar hebben niet altijd genoeg programmeer kennis om deze wijzigingen zelfstandig door te voeren in de software. Voor elke wijziging zijn zij afhankelijk van het IT departement.

tement, waardoor kleine veranderingen vaak vertraging oplopen. Tegelijkertijd is de huidige documentatie niet altijd voldoende nauw verwoven met de code, waardoor het moeilijk is om te achterhalen waar specifieke berekeningen plaatsvinden en welke aannames aan de basis liggen.

De doelgroep van dit onderzoek bestaat dan ook uit twee nauw samenwerkende groepen binnen ILVO: de onderzoekers die de wetenschappelijke modellen ontwikkelen en de IT-professionals die deze modellen implementeren, onderhouden en documenteren. Beide groepen ervaren hinder door de huidige werkwijze: onderzoekers missen autonomie, terwijl IT'ers veel tijd spenderen aan kleine wijzigingen, uitleg en debugging.

Daarom staat in deze bachelorproef de volgende centrale onderzoeksvraag centraal: Hoe kunnen we binnen een .NET-gebaseerde (C# & Razor Pages) onderzoeksapplicatie de kloof verkleinen tussen IT-technische logica en wetenschappelijke analyse, zodat onderzoekers van ILVO zonder diepgaande programmeerkennis inzicht krijgen in de werking van hun intern model en er zelf mee kunnen experimenteren? Deze vraag is relevant omdat ze zowel de transparantie van wetenschappelijke modellen verbetert als de samenwerking tussen onderzoek en IT efficiënter maakt.

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen waarin de wetenschappelijke onderzoekers met hun modellen kunnen experimenteren zonder dat zij een extensieve kennis van programmeren nodig hebben. Hierbij wordt onderzocht welke documentatie vormen het meest geschikt zijn om de interne werking van hun modellen begrijpelijk te maken. Daarnaast wordt nagegaan hoe parameters veilig kunnen worden aangepast. Het uiteindelijke resultaat moet een concreet voorstel en prototype opleveren dat ILVO kan gebruiken om hun onderzoeksmodellen toegankelijker, transparanter en beter onderhoudbaar te maken.

Dit onderzoek wordt bewust afgebakend: het richt zich niet op het herontwerpen van het volledige ILVO-model of het bouwen van een productieklare applicatie, maar wel op het ontwikkelen en evalueren van technieken, workflows en tooling die de interactie tussen code en onderzoek verbeteren. Theoretisch kader wordt gezocht binnen software engineering, documentatieonderzoek, literate programming, domain-driven design en transparantie van wetenschappelijke modellen.

2. Literatuurstudie

2.1. Literate programming

Literate programming („Literate Programming”, g.d.) werd geïntroduceerd door Knuth (1984) als een methode waarbij code en documentatie 1 ge-

ïntegreerd geheel vormen, eerder dan gescheiden artefacten. De bedoeling is dat er 1 bestand is met de code en de documentatie errond. De tools halen de code eruit en creëren een uitvoerbaar bestand en een ander bestand met de documentatie, dat kan een html web pagina zijn, of een $\text{\LaTeX}$ bestand / pdf. Het kan ook een Markdown bestand zijn.

In plaats van ‘code with comments’ vertrekt literate programming vanuit het idee dat een programma in de eerste plaats een verhaal is dat aan een menselijke lezer moet worden uitgelegd, waarbij uitvoerbare code een secundaire afgeleide vorm is (Knuth, 1984). Sindsdien zijn er talrijke systemen ontstaan die Knuths principes toepassen in een moderne ontwikkelingsomgeving.

Hoewel Knuths oorspronkelijke WEB en CWEB nog weinig gebruikt worden in de hedendaagse softwarepraktijk, leven zijn ideeën voort in computationele notebooks, literate programming libraries en research pipelines. Tools zoals Jupyter Notebooks, Org-mode Babel en Quarto implementeren een moderne interpretatie van Knuths ‘weave’ en ‘tangle’-proces, waarin broncode zowel als uitvoerbaar script en als leesbare documentatie dient.

De execution haalt de source code eruit en zorgt voor een uitvoerbaar bestand. De Weave operatie zorgt er voor dat de documentatie uit het source bestand wordt gehaald en die in het gewenste formaat zet. De Tangle operatie verspreidt de source code in de respectievelijke bestanden zelf, niet direct uitvoerbaar.

2.2. Org-mode Babel

Org mode is een ‘major mode’ voor GNU Emacs. Org mode kan voor heel veel gebruikt worden: van notities nemen tot todo-lists, computational notebooks en literate programming (C. Dominik, 2025).

In Org Mode met Babel zijn er ‘src-blocks’:

```
* Src-blocks in org-mode
#+BEGIN_SRC python :results verbatim
    nummer: int = 10
    return nummer
#+END_SRC

#+RESULTS:
: 10
```

Hierin kan code worden uitgevoerd terwijl er tekst rond staat. Dit is het idee van literate programming. De meeste ‘cellen’ in org-mode hebben hun eigen omgeving, maar er bestaan manieren om dit te vermijden. Babel verbetert de src-blocks door een paar zaken te voorzien:

- interactieve en programma executie van code blokken

- code blokken die functies met parameters hebben, kunnen andere code blokken accepteren en oproepen, en
- bestanden exporteren voor literate programming

2.3. Jupyter notebook

Jupyter notebooks zijn gemaakt voor python. Deze notebooks worden door vele mensen gebruikt en in verschillende omgevingen. Dit kan gaan van een student die python leert en zijn notities er bij wil schrijven tot een volledige data analyse door professionele onderzoekers. Dit is een standaard voor computationeel onderzoek en datascience (project jupyter, 2025). Het werkt met verschillende 'cellen': de markdown cellen en de python cellen. De python cellen verbinden met een virtuele omgeving en voeren hierop telkens hun code uit. Aangezien ze werken op dezelfde omgeving zijn de cellen verbonden aan elkaar. Het grootste verschil met org-mode is dat een .org bestand een plain tekst bestand is. Het is mogelijk om dit bestand te bekijken in eender welke editor. Jupyter notebooks niet. Je hebt een editor nodig die jupyter notebooks ondersteunt. Het bekendste is VsCode van Microsoft. Er bestaat ook de officiële command en app van jupyter zelf die een server opstart en dan in de browser beschikbaar is.

Maar jupyter notebooks hebben zeker hun minpunten. A. Rule (2018) tonen aan dat Jupyter Notebooks niet-lineaire uitvoer geschiedenis hebben, wat reproduceerbaarheid ernstig bemoeilijkt. Ook omdat de cellen in een willekeurige volgorde kunnen worden uitgevoerd, moet er opgelet worden met de run volgorde. De notebooks hebben ook een onzichtbare global state. Het zijn json bestanden, dus versie controle beheer kan hier ook moeilijk mee doen. Aangezien dat alles ook in 1 bestand zit, kan het snel 'bloated' zijn.

2.4. RMarkdown

RMarkdown (RStudioPBC, 2020) is ook een computational notebook zoals Jupyter Notebooks, maar het ondersteunt meerdere talen. Het volgt een structuur die sterk aansluit bij de principes van literate programming. Een .Rmd-bestand wordt verwerkt door knitr (Xie Y., 2025), een transparante engine voor het genereren van dynamische rapporten in R, ontwikkeld binnen de software-omgeving voor statistische computing R (The R-Foundation, g.d.).

Knitr voert alle code blokken uit en maakt een nieuw Markdown bestand aan met alle code en hun uitkomsten. Het gemaakte Markdown bestand wordt dan geevalueerd door pandoc (J., 2025), die dan het finale product maakt. Vb:

```
---
title: "viridis demo"
```

```
output: html_document
---
```

```
```{r include - FALSE}
library(viridis)
```
```

The code below ...

```
## Colors
```{r}
image(volcano, col = viridis(200))
```
```

2.5. Tussenbesluit

3. Methodologie

3.1. Probleem- en domeinanalyse

In deze fase wordt onderzocht hoe ILVO momenteel werkt met hun .NET-gebaseerde modellen.

- interviews en overlegmomenten met de onderzoekers (inhoudelijke experts) en de ontwikkelaars
- Analyse van bestaande C#-code, Razor pages, berekeningslogica en interne documentatie
- Identificeren van
 - waar berekeningen plaatsvinden
 - welke parameters en formules onderzoekers willen aanpassen
 - typische fouten of bottlenecks
 - waarvoor onderzoekers nu afhankelijk zijn van IT.

3.2. Literatuurstudie & technologieverkenning

Deze fase biedt het theoretische en technische kader dat rest van het onderzoek ondersteunt.

- Literate programming ((Knuth, 1984), moderne interpretaties in software engineering)
- Reproducible research workflows in wetenschappelijke software
- Documentation-as-code: DocFX, Jupyter notebook, quarto, org mode
- Computational notebooks: jupyter, .NET Interactive, polyglot notebooks
- annotatie- en traceertools binnen codebases
- DSL's en transparantie in wetenschappelijke modellen

3.3. Proof-of-Concept ontwikkeling

Op basis van de analyse en literatuur wordt minstens 1, maar mogelijk meerdere POC's ontwikkeld. Het doel is niet een productieklare applicatie, maar het aantonen van mogelijke oplossingsrichtingen.

Mogelijke richtingen voor POC's zijn:

1. Interactieve notebooks: De formules zijn snel aan te passen en het resultaat is direct te zien. Er is een automatische uitleg van parameters. Er zijn ook visualisaties van impact en gevoeligheidsanalyses
2. Inline annotatie-omgeving van C#-code. De commentaar blokken en uitleg wordt automatisch weergegeven naast de logicablokken. De documentatie gegenereerd via DocFX wordt teruggekoppeld naar de onderzoekers.
3. Model viewer binnen een razor page. Zorgt voor een grafisch overzicht van het model. Mogelijks parameters met sliders. Bedoelt om toekomstige scenario's te simuleren
4. Koppeling tussen code en documentatie. Alles in 1 bestand. Eventueel via Quarto of org-mode.

3.4. Evaluatie en validatie

Er zullen korte gebruikerstesten plaatsvinden:

- Kunnen de onderzoekers formules terugvinden?
- Begrijpen ze de werking van het model beter?
- Kunnen ze parameters aanpassen zonder code te breken?

Semi-gestructureerde interviews met:

- Onderzoekers over bruikbaarheid
- IT'ers over impact op onderhoudbaarheid

Analyse van feedback:

- wat werkt?
- wat werkt niet?
- welke uitbreidingen zijn nodig?

4. Verwacht resultaat, conclusie

4.1. Probleem- en domeinanalyse

Een helder overzicht van de *current flow* (onderzoekers → IT → applicatie)

Een lijst van problemen, zoals:

- beperkte transparantie

- onvoldoende verweven documentatie
- moeilijk traceerbaarheid van berekeningen
- lange feedbackloops

4.2. Literatuurstudie & technologieverkenning

Een lijst van mogelijke technieken en tools om de kloof tussen code en onderzoek te verkleinen. Ook bepalen welke technieken *haalbaar en relevant* zijn voor ILVO.

4.3. Proof-of-Concept ontwikkeling

Een geïntegreerde POC die aantoont hoe onderzoekers:

- parameters kunnen wijzigen,
- resultaten kunnen testen,

zonder afhankelijk te zijn van IT voor kleine experimenten.

4.4. Evaluatie en validatie

Conclusies over welke aanpak het meest effectief is om de kloof te verkleinen en een concreet advies voor ILVO met mogelijke roadmap.

Referenties

- A. Rule, J. D. H., A. Tabard. (2018). Exploration and Explanation in Computational Notebooks [geraadpleegd op 2025-12-11]. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 32, 1–12. <https://doi.org/https://doi.org/10.1145/3173574.3173606>
- C. Dominik, S. E. (2025, oktober 24). *Org mode for Gnu Emacs* [geraadpleegd op 2025-12-10]. Verkregen 24 oktober 2025, van <https://orgmode.org/index.html>
- J., M. (2025). *Pandoc, The universal document converter*. <https://pandoc.org/>
- Knuth, D. E. (1984). *Literate Programming*. *Literate Programming*. (g.d.). Verkregen 10 december 2025, van <http://www.literateprogramming.com/>
- project jupyter. (2025). *Project Jupyter | Home* [geraadpleegd op 2025-12-11]. <https://jupyter.org/>
- RStudioPBC. (2020). *R Markdown* [geraadpleegd op 2025-12-12]. <https://rmarkdown.rstudio.com/>
- TheRFoundation. (g.d.). *R: The Project for Statistical Computing* [Geraadpleegd op 2025-12-12]. <https://www.r-project.org/>
- Xie Y., E. A. (2025). *knitr (Version 1.50)* [Source Code], Geraadpleegd op 2025-12-12]. <https://github.com/yihui/knitr>